WEEK 2 · DAY 2 · OVERVIEW

Confusion Matrix, Precision, and Recall



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 8 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 6 ended with a model probability of default in this file for each of 6,000 validation accounts.

A reviewer cannot work through a probability, only through a list of accounts to look at.

A threshold turns each probability into a flag: 1 at or above it, 0 below.

Today's question is how to score that list.

The file is Day 6's: 30,000 real card accounts in Taiwan in 2005, so nothing here carries over to another market or year.

SEX, **EDUCATION**, **MARRIAGE** and **AGE** are never features: a model using them would sort people by them, and leaving them out does not make a model neutral, because other columns can carry the same information.

A flag means "flagged at threshold t", never a credit decision about anyone.

OBJECTIVES

Turn probabilities into flags, see why accuracy flatters a model on a one-in-five outcome, read the confusion matrix, write one function that scores flags with precision, recall and F1, and watch what moving the threshold trades.

ABOUT 45 MIN WITH EXERCISES / 8 SECTIONS / 4 EXERCISES

Your Route Through Day 2

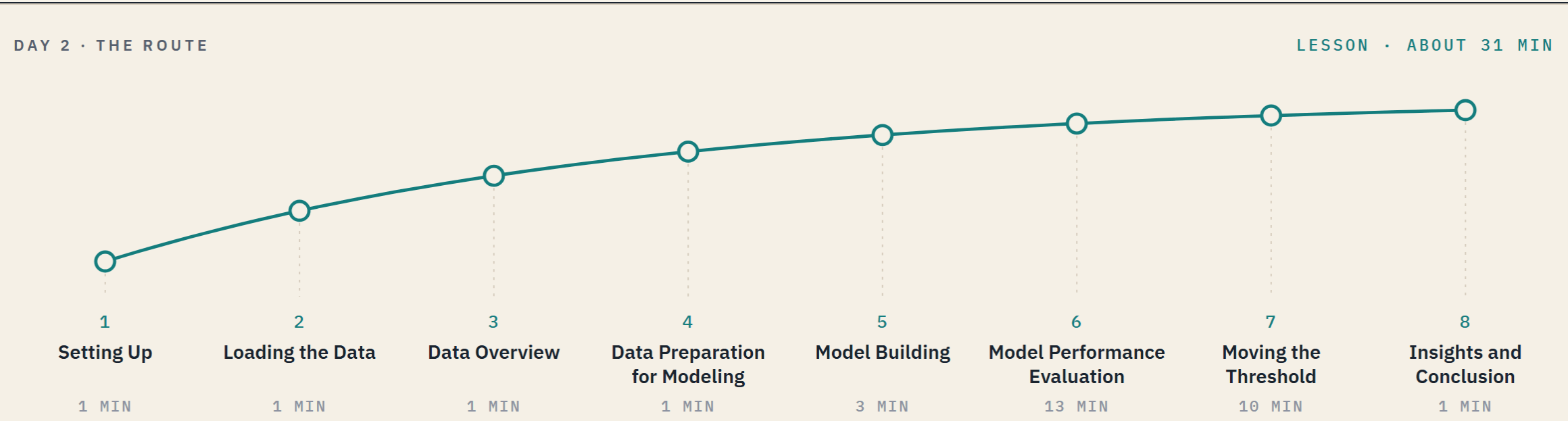


FIGURE 1 Eight sections, in order, with about how long each takes.

Time. About 31 minutes for the lesson, then about 12 for the four exercises.

Setting Up

FIX THE SEED AND FIND THE DATA

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

ONE LOOK FOR EVERY CHART

```
sns.set_theme(style="whitegrid")
sns.set_palette("colorblind")
pd.set_option("display.float_format", "{:,.4f}".format)
CELL_COLOURS = {"TN": "#56B4E9", "FP": "#DE8F05", "FN": "#D55E00", "TP": "#0173B2"}
```

Run this cell and move on: it fixes the seed and finds the course data.

Loading the Data

READ THE FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
```

ONE ROW PER ACCOUNT

- **`ID`**: a row label, never a feature
- **`LIMIT_BAL`**: the credit limit, NT dollars
- **`PAY_0`**, **`PAY_2`** to **`PAY_6`**: repayment status, September back to April
- **`BILL_AMT1`** to **`BILL_AMT6`**: the bill, NT dollars
- **`PAY_AMT1`** to **`PAY_AMT6`**: the amount paid, NT dollars
- **`SEX`**, **`EDUCATION`**, **`MARRIAGE`**, **`AGE`**: about the person, never features
- **`default payment next month`**: 1 if the account defaulted the following month, 0 if not, **the target**

Data Overview

THE SIZE OF THE FILE, AND ITS DEFAULTS

```
print(card.shape)
print(card[TARGET].sum(), "accounts defaulted:", round(card[TARGET].mean() * 100, 2), "percent")
```

```
(30000, 25)
6636 accounts defaulted: 22.12 percent
```

WHAT THE FILE IS

The file is Day 6's: 30,000 real card accounts in Taiwan in 2005, so nothing here carries over to another market or year.

The file is Day 6's and passed Day 6's checks, so we go straight to the split.

Data Preparation for Modeling

CHOOSE THE FEATURES

```
PERSON_COLUMNS = ["SEX", "EDUCATION", "MARRIAGE", "AGE"]
FEATURES = [column for column in card.columns if column not in ["ID", TARGET] + PERSON_COLUMNS]
print(len(FEATURES), "features")
```

```
19 features
```

CUT THREE SETS

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Defaults in validation:", y_valid.sum(), " Default rate:", round(y_valid.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Defaults in validation: 1327  Default rate: 0.2212
```

Fit on training accounts, score on validation accounts; the test accounts stay closed until Day 10.

Model Building

A PROBABILITY FOR EVERY ACCOUNT

	DEFAULTED	MODEL PROBABILITY
18316	1	0.2511
1482	0	0.1432
10396	1	0.0557
9360	0	0.1018

A FLAG AT 0.5

```
flag_valid = (proba_valid >= 0.5).astype(int)
print("Flagged:", flag_valid.sum(), "of", len(flag_valid), "accounts,", round(flag_valid.mean() * 100, 2), "percent")
```

```
Flagged: 447 of 6000 accounts, 7.45 percent
```

Account 18316 defaulted with a model probability of 0.2511; account 10396 defaulted at 0.0557.

Model Performance Evaluation

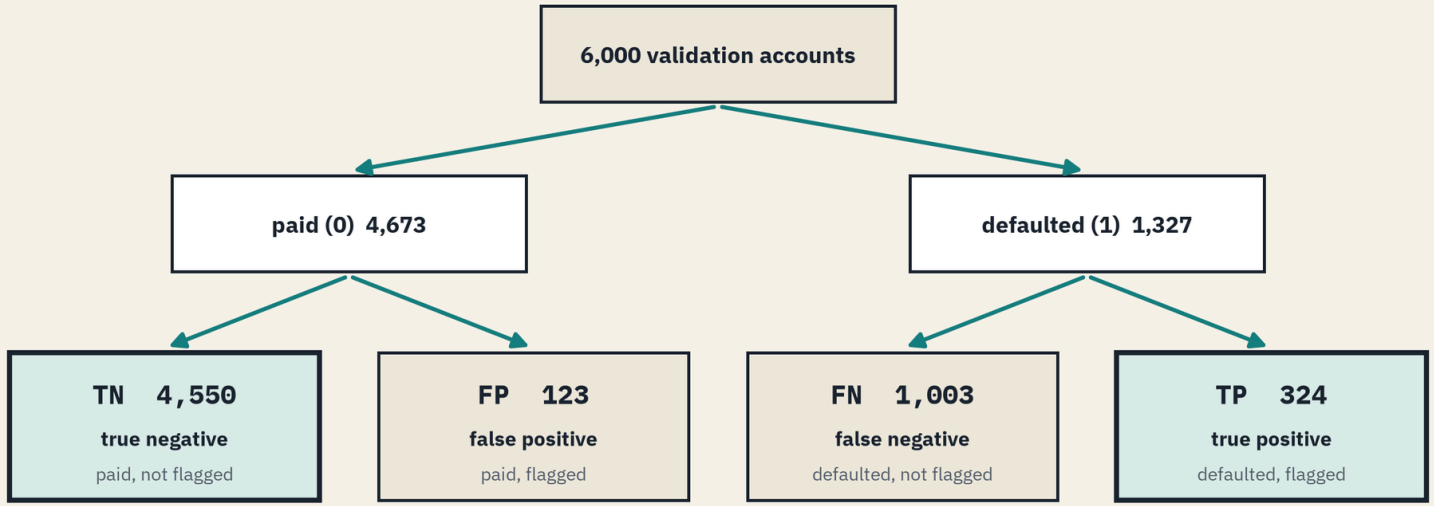


FIGURE 2 The 6,000 validation accounts split by outcome, then by the flag at 0.5: TN 4,550, FP 123, FN 1,003 and TP 324.

COUNT THE FOUR KINDS

```
confusion_matrix(y_valid, flag_valid)

array([[4550,  123],
       [1003,  324]])
```

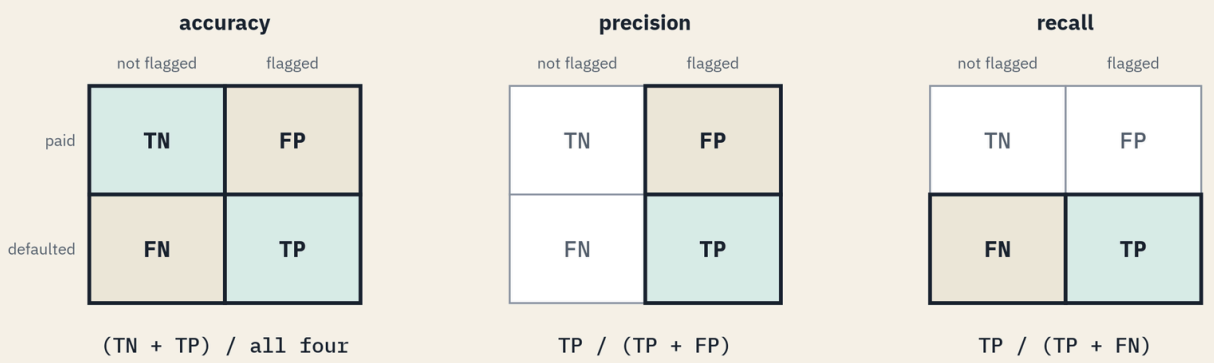
NAME THE ROWS AND COLUMNS

	NOT FLAGGED (0)	FLAGGED (1)
paid (0)	4550	123
defaulted (1)	1003	324
total	5553	447

1,327 accounts defaulted and 447 were flagged, but only 324 are both.

Precision, Recall and F1

Which cells each measure reads



(TN + TP) / all four

TP / (TP + FP)

TP / (TP + FN)

Teal: the numerator. Shaded and outlined: every cell the measure reads.

FIGURE 3 Accuracy reads all four cells, precision the flagged column and recall the defaulted row, each with TP on top.

THE THREE MEASURES

```
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

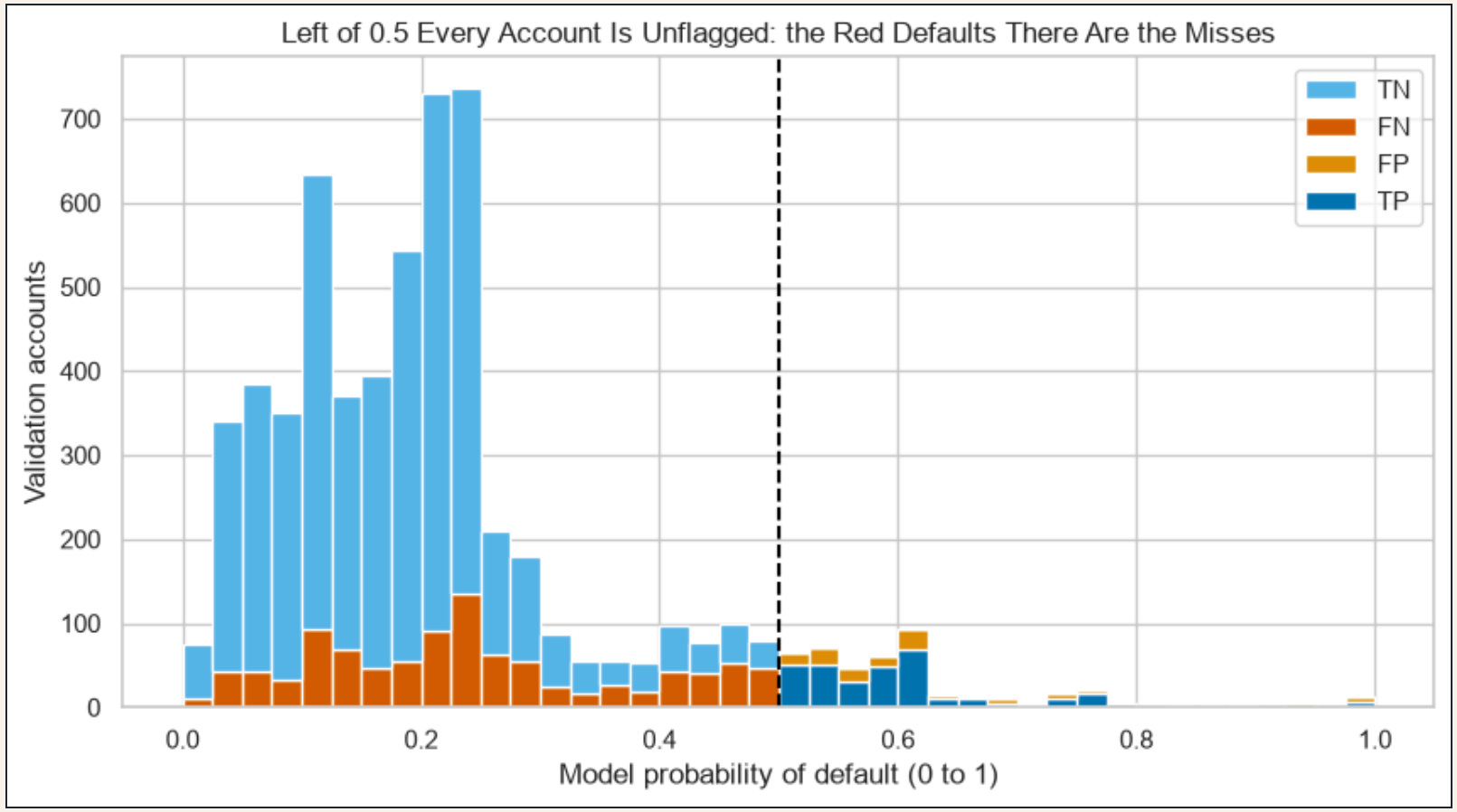
Precision: 0.7248  Recall: 0.2442  F1: 0.3653
```

TRAINING AGAINST VALIDATION

	TRAINING	VALIDATION
Accuracy	0.8131	0.8123
Recall	0.2557	0.2442
Precision	0.7179	0.7248
F1	0.3770	0.3653

The defaulted column is where a model earns its keep, and two ratios read it.

Moving the Threshold



NAME EVERY ACCOUNT'S CELL

```
TN    4550
FN    1003
TP     324
FP     123
Name: count, dtype: int64
```

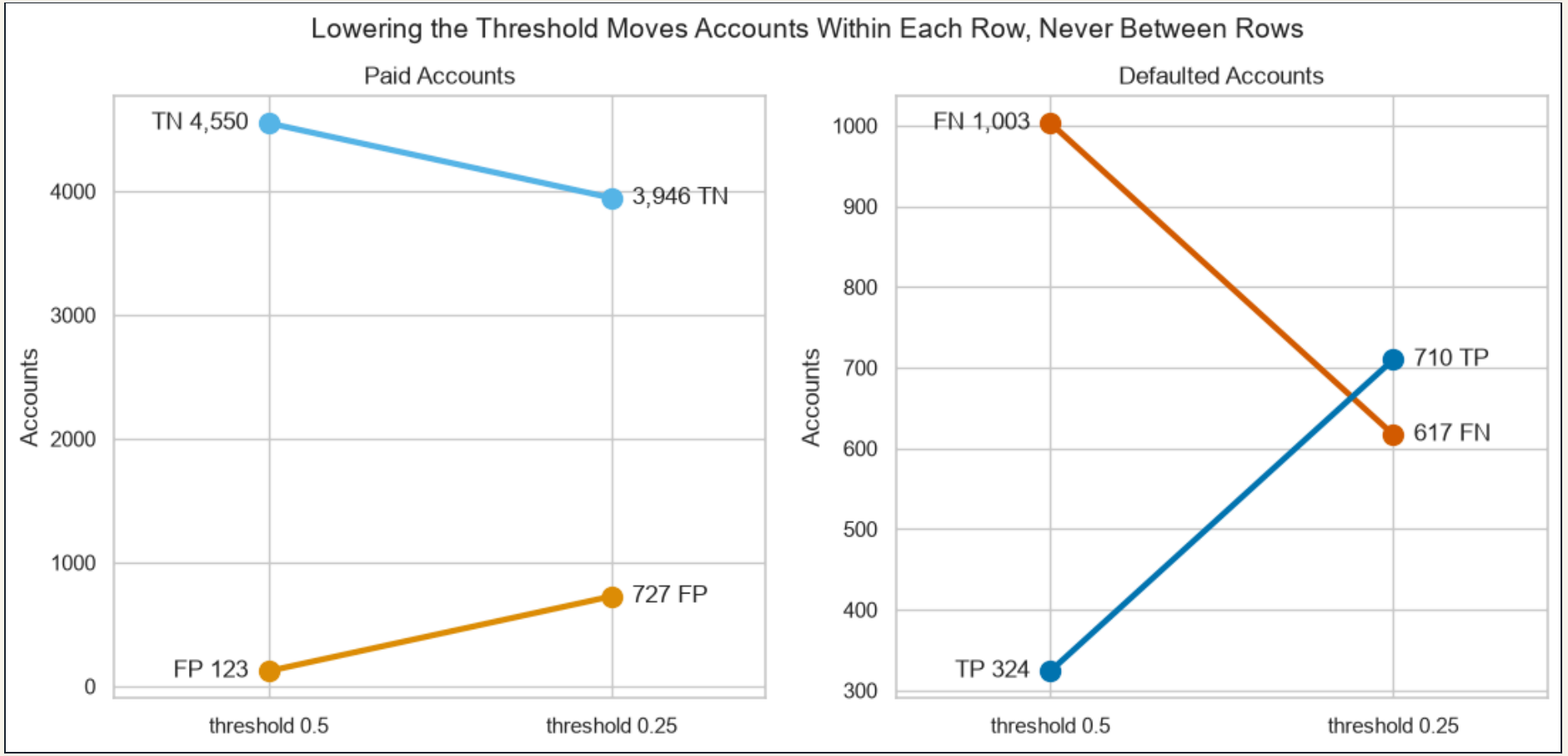
The same four counts as the matrix, now one name per account, so a chart can colour each account by its cell.

READING THE CHART

Left of the line sit TN and the 1,003 FN; right of it, FP and TP.

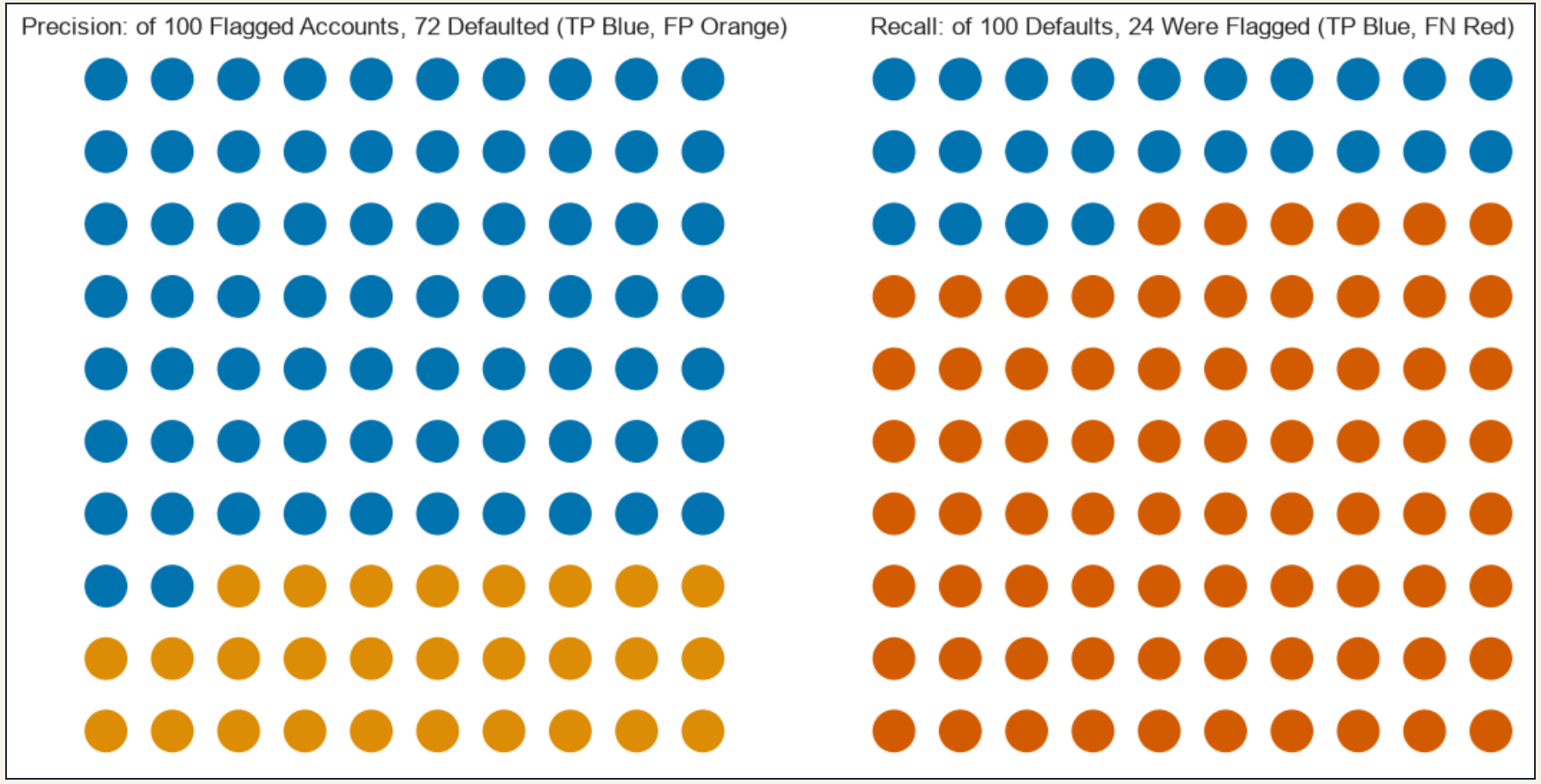
Move the line left and FN turns into TP, but TN also turns into FP.

From 0.5 to 0.25, Cell by Cell



Each row's total stays put: a threshold changes the flag, never what happened.

Insights and Conclusion



A flag is right in two ways and wrong in two; count all four before trusting one number.

You Can Now

- Turn probabilities into flags at a threshold.
- Lay out and read a confusion matrix.
- Compute precision, recall and F1 by hand and with scikit-learn.
- Score any classifier with **`model_performance_classification`**, against baselines.
- Say what moving the threshold trades, and read **`classification_report`**.

NEXT

WEEK 2 · DAY 3

ROC Curves and Choosing a Threshold

Day 8 draws every threshold at once and chooses one on validation accounts.

